

Задача 1

$$M_1(x) = x^4 + x + 1, \quad M_3(x) = x^4 + x^3 + x^2 + x + 1.$$

Поле $\text{GF}(2^4)$ задано примитивным полиномом $p(x) = x^4 + x + 1$, $\alpha^4 = \alpha + 1$.

1. Порождающий полином

$$g(x) = M_1(x) M_3(x) = x^8 + x^7 + x^6 + x^4 + 1.$$

2. Систематическая порождающая матрица G

Матрица $G = [I_7 \mid P]$, где столбцы P – остатки от деления x^{8+i} на $g(x)$ ($i = 0 \dots 6$). Численно получено:

$$G = \left(\begin{array}{cccccccc|cccccccc} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 \end{array} \right).$$

3. Проверочная матрица H и условие $GH^T = 0$

Для систематической $G = [I \mid P]$ численно полученная проверочная матрица $H = [P^T \mid I_8]$:

$$H = \left(\begin{array}{cccccccc|cccccccc} 1 & 0 & 1 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{array} \right).$$

Произведение GH^T по модулю 2, полученное численно, есть нулевая матрица 7×8 . $GH^T = 0$.

Код на Python

```
1 import numpy as np
2
3 exp = [0] * 15
4 log = [0] * 16
5 val = 1
6 for i in range(15):
7     exp[i] = val
8     log[val] = i
9     val <<= 1
10    if val & 16:
11        val ^= 0x13
12    val &= 0xF
13
14 def poly_divmod_int(dividend, divisor):
15     quot, rem = 0, dividend
16     d_deg = divisor.bit_length() - 1
17     while rem.bit_length() > d_deg:
18         shift = rem.bit_length() - d_deg - 1
19         if (rem >> (d_deg + shift)) & 1:
20             rem ^= divisor << shift
21         quot ^= 1 << shift
22     return quot, rem
23
24 n, k = 15, 7
25 g = 0b111010001
```

```

26
27 P = np.zeros((k, n-k), dtype=int)
28 for i in range(k):
29     rem = poly_divmod_int(1 << (8 + i), g)[1]
30     row_bin = f"{rem:0{n-k}b}"
31     P[i, :] = [int(b) for b in row_bin]
32
33 G = np.hstack((np.eye(k, dtype=int), P))
34 print("Systematic generator matrix G (rows):")
35 for row in G:
36     print(' '.join(str(int(b)) for b in row))
37
38 H = np.hstack((P.T, np.eye(n-k, dtype=int)))
39 print("\nParity-check matrix H:")
40 for row in H:
41     print(' '.join(str(int(b)) for b in row))
42
43 prod = (G @ H.T) % 2
44 print("\nProduct G * H^T (mod 2):")
45 print(prod)
46 if np.all(prod == 0):
47     print("Condition G * H^T = 0 satisfied.")
48 else:
49     print("Error: product is not zero.")

```